DEBUGGING

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 4: Land Every Fix With a Failing Test and Invariant

- Before accepting any AI-proposed patch, require two things alongside the patch:
- 1. A test that currently fails on the buggy code for the right reason.
- 2. An executable assertion of the invariant inside the function that should hold after the fix.
- The patch must make the test pass and preserve the invariant under adversarial inputs.
- 'The model says it works' is not evidence.

Guideline 4: Reasoning

- Automated program repair has a known failure mode: a patch passes every test but is still wrong because the suite did not pin the root cause (the overfitting problem)
- A large fraction of plausible APR patches are semantically incorrect once inspected against a ground-truth specification
- A patch is only accepted after a hypothesis has been confirmed against runtime evidence
- LLMs frequently invent APIs or confuse versions, so a fix that reads as sensible can reference functions that do not exist
- The invariant upgrades a test-only regimen toward Design-by-Contract: tests pin boundary behavior, invariants pin what must be true inside the function

Guideline 4: Key Practices - Prompting

- Ask for the failing test, the invariant, and the minimal patch in that order, not just the patch.
- Ask for two adversarial tests that a naive fix (one that special-cases the failing input) would not pass.
- Give the AI your language and library version numbers to reduce hallucinated API calls.

Guideline 4: Key Practices - Reviewing

- Run the failing test on the unpatched code and confirm it fails for the right reason by reading the assertion, not just the red dot.
- Apply the patch, re-run the whole suite, and run the adversarial tests.
- Read the invariant assertion. Can you name an input that would violate it? If yes, add that input to the adversarial set.
- Watch for overfit patterns: patch special-cases the exact test input, or wraps failing code in a broad try/except that swallows the symptom.

Guideline 5: Ask the AI to Classify Its Fix

- Before reading the diff of any AI-proposed patch, ask the AI one question:
- **'Does this fix remove the cause from the execution trace, or does it add a guard that prevents the symptom from being observed?'**
- A root-cause fix changes the code so the wrong state is never reached.
- A symptom fix adds a guard (a None check, a try/except, a default return) that intercepts the failure after the wrong state has already been produced.
- Always know which kind of fix you are accepting.

Guideline 5: Reasoning

- Most plausible APR patches suppress a symptom rather than remove a cause; hard to detect from test results alone
- LLM performance drops sharply on semantic bugs compared to syntactic ones
- Semantic bugs are where symptom fixes are most tempting: wrong state is subtle, root cause is hard to articulate
- A fix is valid only if it removes the cause from the execution trace, not just prevents the observable failure
- Developers stop at the first plausible fix; a confident AI patch makes this failure mode worse

Guideline 5: Key Practices - Before You Read the Diff

- Ask the AI to classify its own fix in one sentence before you evaluate any code: root-cause removal or symptom guard?
- If it says guard, ask it to describe the root cause the guard is protecting against and why that cause is not being fixed directly.
- If the AI cannot clearly state which category its fix falls into, treat that as a signal the reasoning is unclear; ask it to re-examine before you proceed.

Guideline 5: Key Practices - Reading the Diff and After

- Recognize four common symptom-fix patterns: broad try/except, None guard just before the failure, special-case branch for the exact input, logging-and-continue.
- For each yellow-flag hunk, trace backward: if the guard tests 'if result is None', ask what code path makes result None.
- Run a probe input that exercises the same root cause via a different call path than the one in your failing test. A symptom guard will still fail this probe.
- If you accepted a symptom fix intentionally, add a comment naming the root cause and linking to the tracking issue.

Guideline 6: Keep a Debugging Log

- Write down every hypothesis tested, every experiment run, and every fix tried, with the result and a one-line reason for why you picked it.
- Feed the log back in with each prompt so the next suggestion is constrained by what has already been ruled out.
- When the conversation starts looping, start a fresh chat with only the log and your current minimal reproducer as context.

Guideline 6: Reasoning

- The debugging logbook keeps the cause-effect chain visible and prevents repeated experiments
- Losing track of what has been tried is a main reason professional debugging sessions spiral into wasted hours
- Multi-turn AI debugging gets stuck because the model repeatedly proposes fixes already tried earlier in the same thread
- Re-pasting a short log at each turn turns the model's limited memory into a non-issue
- The 'why I picked this' notes become a record of your own learning: the actual educational payoff

Guideline 6: Key Practices - Prompting

- Start the prompt with a short log: hypotheses tried and ruled out (with reasons), picks and reasons, fixes tried and results.
- Ask the AI to reason from the log: 'Given what has been ruled out, what is the next most likely hypothesis and cheapest experiment?'
- If the chat is more than a handful of turns long, start a new chat with only the log and current minimal reproducer. Do not drag the old transcript.

Guideline 6: Key Practices - Reviewing

- If the AI suggests something already on your tried list, point it out explicitly. Do not let the same wrong suggestion get re-laundered through prose.
- Update the log immediately after each experiment or fix attempt, while the result is fresh. Delayed log updates are worse than no log.
- When handing off the bug (to a teammate, TA, or future-you), share the log, not the chat transcript. It is the actual record of what you learned.

References

- [1] Layman et al. Debugging Revisited: Toward Understanding the Debugging Needs of Contemporary Software Developers. ESEM 2013. DOI: 10.1109/ESEM.2013.43
- [2] Bajpai et al. Let's Fix this Together: Conversational Debugging with GitHub Copilot. VL/HCC 2024. IEEE: 10714559
- [3] Kang et al. Explainable Automated Debugging via LLM-Driven Scientific Debugging. Empir. Softw. Eng. 2025. DOI: 10.1007/s10664-024-10594-x
- [4] Chen et al. Teaching Large Language Models to Self-Debug. arXiv:2304.05128. ICLR 2024.
- [5] Perry et al. Do Users Write More Insecure Code with AI Assistants? CCS 2023. DOI: 10.1145/3576915.3623157

References (continued)

- [6] Zeller. Why Programs Fail: A Guide to Systematic Debugging, 2nd ed. Morgan Kaufmann, 2009.
- [7] Yang et al. Patch Correctness Assessment: A Survey. ACM TOSEM 2024. DOI: 10.1145/3702972
- [8] Liu et al. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. arXiv:2404.00971
- [9] Kang, An, Yoo. A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization. FSE 2024. DOI: 10.1145/3660771
- [10] Zeller and Hildebrandt. Simplifying and Isolating Failure-Inducing Input. IEEE TSE 2002. DOI: 10.1109/32.988498

References (continued)

- [11] Weiser. Program Slicing. IEEE TSE 1984. DOI: 10.1109/TSE.1984.5010248
- [12] Zhong et al. Debug like a Human: LLM Debugger via Verifying Runtime Execution. ACL Findings 2024. DOI: 10.18653/v1/2024.findings-acl.49
- [13] Meyer. Applying Design by Contract. IEEE Computer 1992. DOI: 10.1109/2.161279
- [14] Parnin and Orso. Are Automated Debugging Techniques Actually Helping Programmers? ISSTA 2011. DOI: 10.1145/2001420.2001445
- [15] Tian et al. DebugBench: Evaluating Debugging Capability of LLMs. ACL Findings 2024. DOI: 10.18653/v1/2024.findings-acl.247